36

EXOTICA

In this, the final chapter of our journey, we will look at some odds and ends. While we have certainly covered a lot of ground in the previous chapters, there are many bash features that we have not covered. Most are fairly obscure and useful mainly to those integrating bash into a Linux distribution. However, there are a few that, while not in common use, are helpful for certain programming problems. We will cover them here.

Group Commands and Subshells

bash allows commands to be grouped together. This can be done in one of two ways, either with a *group command* or with a *subshell*. We introduced group commands back in Chapter 6, and as we recall, a group command uses this syntax:

```
{ command1; command2; [command3; ...] }
```

Subshells use a similar syntax.

```
(command1; command2; [command3;...])
```

A group command surrounds its commands with braces and a subshell uses parentheses. It is important to note that, because of the way bash implements group commands, the braces must be separated from the commands by a space and the last command must be terminated with either a semicolon or a newline prior to the closing brace.

So, what are group commands and subshells good for? While they have an important difference (which we will get to in a moment), they both can be used to manage redirection. Let's consider a script segment that performs redirection on multiple commands.

```
ls -l > output.txt
echo "Listing of foo.txt" >> output.txt
cat foo.txt >> output.txt
```

This is pretty straightforward. Three commands have their output redirected to a file named *output.txt*. Using a group command, we could code this as follows:

```
{ ls -l; echo "Listing of foo.txt"; cat foo.txt; } > output.txt
```

Using a subshell is similar.

```
(ls -l; echo "Listing of foo.txt"; cat foo.txt) > output.txt
```

Using this technique we have saved ourselves some typing, but where a group command or subshell really shines is with pipelines. When constructing a pipeline of commands, it is often useful to combine the results of several commands into a single stream. Group commands and subshells make this easy.

```
{ ls -l; echo "Listing of foo.txt"; cat foo.txt; } | lpr
```

Here we have combined the output of our three commands and piped them into the input of lpr to produce a printed report.

While group commands and subshells look similar and can both be used to combine streams for redirection, there is an important difference between the two. Whereas a group command executes all of its commands

in the current shell, a subshell (as the name suggests) executes its commands in a child copy of the current shell. This means the environment (including things like shell functions, aliases, and variables) is copied and given to a new instance of the shell. This copy of the environment is different from the way a child process ordinarily works in that the subshell inherits an entire copy of the parent's environment, whereas a child process inherits only the exported variables from the parent shell. When the subshell exits, its copy of the environment is lost, so any changes made to the subshell's environment (including variable assignments) are lost as well.

Most importantly, subshells, like child processes and unlike group commands, cannot alter the parent shell's environment. Let's demonstrate; first with a group command:

```
[me@linuxbox ~]$ foo="Original Value"
[me@linuxbox ~]$ { foo="Altered Value"; echo $foo; }
Altered Value
[me@linuxbox ~]$ echo $foo
Altered Value
```

Next, we'll do the same steps with a subshell.

```
[me@linuxbox ~]$ foo="Original Value"
[me@linuxbox ~]$ ( foo="Altered Value"; echo $foo )
Altered Value
[me@linuxbox ~]$ echo $foo
Original Value
```

As we can see, the group command is able to modify the value of `foo` in the current shell while the subshell cannot. This characteristic of subshells is handy when we want to do something like change directories. When we perform a `cd` command in a subshell, the current working directory is altered for the duration of the subshell, but after returning to the parent shell, the current working directory is unchanged.

```
[me@linuxbox ~]$ pwd
/home/me
[me@linuxbox ~]$ ( cd /usr/bin; pwd )
/usr/bin
[me@linuxbox ~]$ pwd
/home/me
```

In general, however, unless a script requires a subshell, group commands are preferable to subshells. Group commands are both faster and require less memory.

In the script that follows, we will use group commands and look at several programming techniques that can be employed in conjunction with associative arrays. This script, called `array-2`, when given the name of a directory, prints a listing of the files in the directory along with the names of the file's owner and group owner. At the end of the listing, the script prints a tally of the number of files belonging to each owner and group.

Here we see the results (condensed for brevity) when the script is given the directory */usr/bin*:

```
[me@linuxbox ~]$ array-2 /usr/bin
/usr/bin/2to3-2.6                       root       root
/usr/bin/2to3                           root       root
/usr/bin/a2p                            root       root
/usr/bin/abrowser                       root       root
/usr/bin/aconnect                       root       root
/usr/bin/acpi_fakekey                   root       root
/usr/bin/acpi_listen                    root       root
/usr/bin/add-apt-repository             root       root
.
.
.
/usr/bin/zipgrep                        root       root
/usr/bin/zipinfo                        root       root
/usr/bin/zipnote                        root       root
/usr/bin/zip                            root       root
/usr/bin/zipsplit                       root       root
/usr/bin/zjsdecode                      root       root
/usr/bin/zsoelim                        root       root

File owners:
daemon    :     1 file(s)
root      :  1394 file(s)

File group owners:
crontab   :     1 file(s)
daemon    :     1 file(s)
lpadmin   :     1 file(s)
mail      :     4 file(s)
mlocate   :     1 file(s)
root      :  1380 file(s)
shadow    :     2 file(s)
ssh       :     1 file(s)
tty       :     2 file(s)
utmp      :     2 file(s)
```

Here is a listing (with line numbers) of the script:

```
 1  #!/bin/bash
 2
 3  # array-2: Use arrays to tally file owners
 4
 5  declare -A files file_group file_owner groups owners
 6
 7  if [[ ! -d "$1" ]]; then
 8      echo "Usage: array-2 dir" >&2
 9      exit 1
10  fi
11
12  for i in "$1"/*; do
13      owner="$(stat -c %U "$i")"
```

```
14        group="$(stat -c %G "$i")"
15        files["$i"]="$i"
16        file_owner["$i"]="$owner"
17        file_group["$i"]="$group"
18        ((++owners[$owner]))
19        ((++groups[$group]))
20    done
21
22    # List the collected files
23    { for i in "${files[@]}"; do
24        printf "%-40s %-10s %-10s\n" \
25            "$i" "${file_owner["$i"]}" "${file_group["$i"]}"
26    done } | sort
27    echo
28
29    # List owners
30    echo "File owners:"
31    { for i in "${!owners[@]}"; do
32        printf "%-10s: %5d file(s)\n" "$i" "${owners["$i"]}"
33    done } | sort
34    echo
35
36    # List groups
37    echo "File group owners:"
38    { for i in "${!groups[@]}"; do
39        printf "%-10s: %5d file(s)\n" "$i" "${groups["$i"]}"
40    done } | sort
```

Let's take a look at the mechanics of this script:

Line 5

Associative arrays must be created with the `declare` command using the `-A` option. In this script, we create five arrays as follows:

- `files` contains the names of the files in the directory, indexed by filename.
- `file_group` contains the group owner of each file, indexed by filename.
- `file_owner` contains the owner of each file, indexed by filename.
- `groups` contains the number of files belonging to the indexed group.
- `owners` contains the number of files belonging to the indexed owner.

Lines 7–10

These lines check to see that a valid directory name was passed as a positional parameter. If not, a usage message is displayed and the script exits with an exit status of 1.

Lines 12–20

These lines loop through the files in the directory. Using the `stat` command, lines 13 and 14 extract the names of the file owner and group owner and assign the values to their respective arrays (lines 16 and 17) using the name of the file as the array index. Likewise, the filename itself is assigned to the `files` array (line 15).

Lines 18–19

The total number of files belonging to the file owner and group owner are incremented by one.

Lines 22–27

The list of files is output. This is done using the "${*array*[@]}" parameter expansion, which expands into the entire list of array elements with each element treated as a separate word. This allows for the possibility that a filename may contain embedded spaces. Also note that the entire loop is enclosed in braces, thus forming a group command. This permits the entire output of the loop to be piped into the `sort` command. This is necessary because the expansion of the associative array elements is not sorted.

Lines 29–40

These two loops are similar to the file list loop except that they use the "${!*array*[@]}" expansion, which expands into the list of array indexes rather than the list of array elements.

Process Substitution

We saw an example of the subshell environment problem in Chapter 28 when we discovered that a `read` command in a pipeline does not work as we might intuitively expect. To recap, if we construct a pipeline as follows, then the content of the `REPLY` variable is always empty because the `read` command is executed in a subshell, and its copy of `REPLY` is destroyed when the subshell terminates:

```
echo "foo" | read
echo $REPLY
```

Because commands in pipelines are always executed in subshells, any command that assigns variables will encounter this issue. Fortunately, the shell provides an exotic form of expansion called *process substitution* that can be used to work around this problem.

Process substitution is expressed in two ways. For processes that produce standard output, it looks like this:

```
<(list)
```

Or, for processes that intake standard input, it looks like this:

```
>(list)
```

list is a list of commands.

To solve our problem with read, we can employ process substitution like this:

```
read < <(echo "foo")
echo $REPLY
```

Process substitution allows us to treat the output of a subshell as an ordinary file for the purposes of redirection. In fact, since it is a form of expansion, we can examine its real value.

```
[me@linuxbox ~]$ echo <(echo "foo")
/dev/fd/63
```

By using echo to view the result of the expansion, we see that the output of the subshell is being provided by a file named */dev/fd/63*.

Process substitution is often used with loops containing read. Here is an example of a read loop that processes the contents of a directory listing created by a subshell:

```
#!/bin/bash

# pro-sub: demo of process substitution

while read -r attr links owner group size date time filename; do
    cat << _EOF_
Filename:   $filename
Size:       $size
Owner:      $owner
Group:      $group
Modified:   $date $time
Links:      $links
Attributes: $attr

_EOF_
done < <(ls -l --time-style="+%F %H:%m"| tail -n +2)
```

The loop executes read for each line of a directory listing. The listing itself is produced on the final line of the script. This line redirects the output of the process substitution into the standard input of the loop. The tail command is included in the process substitution pipeline to eliminate the first line of the listing, which is not needed.

When executed, the script produces output like this:

```
[me@linuxbox ~]$ pro-sub | head -n 20
Filename:   addresses.ldif
```

```
Size:       14540
Owner:      me
Group:      me
Modified:   2009-04-02 11:12
Links:      1
Attributes: -rw-r--r--

Filename:   bin
Size:       4096
Owner:      me
Group:      me
Modified:   2009-07-10 07:31
Links:      2
Attributes: drwxr-xr-x

Filename:   bookmarks.html
Size:       394213
Owner:      me
Group:      me
```

Constructing Commands with eval

The `eval` builtin is a strange and mysterious command. Simply described, it takes a list of arguments, combines them into a string, and passes it to the shell to execute. So the question naturally becomes, what's this for? Why would we use this?

There are certain cases in shell scripting where we need to construct a command dynamically at runtime, and `eval` allows us to do that. Let's perform an experiment. While we didn't cover it explicitly, it's possible to place a command in a string variable and then rely on parameter expansion to expand the variable into a command.

```
[me@linuxbox ~]$ cmd="echo foo"
[me@linuxbox ~]$ $cmd
foo
[me@linuxbox ~]$
```

Now let's look at what happens when we include a variable in our command.

```
[me@linuxbox ~]$ str=abcde
[me@linuxbox ~]$ cmd="echo $str"
[me@linuxbox ~]$ $cmd
abcde
[me@linuxbox ~]$
```

That does what we expect, but let's see what happens when we assign a new value to str.

```
[me@linuxbox ~]$ str=ABCDE
[me@linuxbox ~]$ $cmd
abcde
[me@linuxbox ~]$
```

The results didn't change. This makes sense because we already performed parameter expansion on $str when we assigned cmd. But what if we wanted to have the variable expanded later? We could enclose our command in single quotes to suppress parameter expansion during the initial variable assignment.

```
[me@linuxbox ~]$ cmd='echo $str'
[me@linuxbox ~]$ $cmd
$str
[me@linuxbox ~]$
```

Though we didn't get the parameter expansion during the assignment of cmd, we didn't get it when we executed cmd either. The reason for this is the shell performs expansion only once even when the expansion ($cmd) results in something that could be further expanded ($str).

If we use eval, we get the additional level of expansion.

```
[me@linuxbox ~]$ str=abcde
[me@linuxbox ~]$ cmd='echo $str'
[me@linuxbox ~]$ eval "$cmd"
abcde
[me@linuxbox ~]$ str=ABCDE
[me@linuxbox ~]$ eval "$cmd"
ABCDE
[me@linuxbox ~]$
```

This shows us what eval does. It joins its arguments into a single string; performs tilde, parameter, and pathname expansion on the contents of the string; and then passes the string to the current shell (no subshell is created) for execution.

BE CAREFUL WITH EVAL

The eval command has a bad reputation. This stems from concern over how easy it is to add security vulnerabilities to a shell script with eval. If the string given to eval comes from an external source (that is, user input), care must be taken to ensure that there are no unauthorized commands embedded in the string (called a *code injection attack*). Always verify the contents of the string given to eval.

A Wordle Helper

In the script that follows, we will use `eval` to help find possible answers to a *Wordle* puzzle.

For those unfamiliar with this popular game, Wikipedia describes it this way:

> Wordle is a web-based word game created and developed by Welsh software engineer Josh Wardle. Players have six attempts to guess a five-letter word, with feedback given for each guess in the form of colored tiles indicating when letters match or occupy the correct position.

Basically, when we make a guess the game will tell us if any of the letters in the guess are present in the secret word and if a letter in the guess matches the position in the secret word. So each time we make a guess, we learn more about the secret word until we (ideally) can figure out its identity. The goal of the game is to guess the word in the fewest number of tries.

The helper script outputs a list of words that match what is known from the guesses so far. To do this, the script is given a simple regular expression that tells the script what letters are known and in what positions they occupy in the secret word. In addition, we provide a list of letters that are known to be contained within the secret word, and letters that are known not to appear in the secret word.

Let's say that the secret word is *about* and we guessed *bloat*. *Wordle* would tell us the *o* and *t* appear in the word in their respective positions and that the word also contains an *a* and a *b* in unknown positions, and further, the secret word does not contain an *l*. In this situation, we would invoke the script this way:

```
[me@linuxbox ~]$ eval-wordle ..o.t +a +b -l
```

The first argument is a five-character regular expression with the *o* and *t* in their respective positions. Next are the known characters present in the secret word and the letters known not to be the secret word. The script responds with the following, showing that only two words in the dictionary meet the selection criteria:

```
abort
about
2
```

The script works by filtering a dictionary (*/usr/share/dict/words*) according to the selection criteria and then displaying what's left. To do this, we need to construct a long multipart pipeline that varies in length depending on how many positional parameters are provided on the command line. By doing it this way, we eliminate the need for temporary files to hold intermediate results.

```
1    #!/bin/bash

2    # eval-wordle - demonstrate eval by solving wordle puzzles
```

```
3     PROGNAME=${0##*/}
4     DICTIONARY=/usr/share/dict/words

5     usage() {
6       printf "%s\n" "Usage: ${PROGNAME} [-h|--help]"
7       printf "%s\n" "       ${PROGNAME} regex +-char..."
8     }

9     help_message() {
10      cat << _EOF_
11    $PROGNAME - Wordle Helper

12    $(usage)

13    Options:
14      -h, --help    Display this help message and exit.

15    Arguments:

16      The first argument must be a five character regular
17      expression at minimum of '.....' representing five
18      unknown characters in the answer. This is followed by
19      zero or more character known to be either present or
20      absent from the answer. These are expressed as either
21      +char for letters known to be present or -char for
22      letters known to not be in the answer.

23    Example:

24      $PROGNAME a.... +o -v

25      This means we know that the answer starts with 'a' in
26      the first postion and also contains an 'o' but does
27      not contain a 'v'.

    _EOF_

28      return
29    }

30    add_plus() { # Add a letter
31      local char="$1"

32      echo " | grep $char"
33      return
34    }

35    add_minus() { # Sutraact a letter
36      local char="$1"

37      echo " | grep -v $char"
38      return
39    }
```

```
40    # Parse command-line

41    if [[ "$1" == "-h" || "$1" == "--help" ]]; then
42      help_message
43      exit 0
44    fi

45    if (( "${#1}" == 5 )); then
49      known_chars="$1"
50      shift
48    else
46      echo "First argument must be a 5 character regex." >&2
47      exit 1
51    fi

52    cmd="LANG=POSIX grep '^.....$' $DICTIONARY \
53      | grep -v '[[:punct:]]' \
54      | grep -v '[[:upper:]]' \
55      | grep '$known_chars'"

56    while [[ -n "$1" ]]; do
57      case "$1" in
58        -[[:alpha:]])
59          cmd="${cmd}$(add_minus "${1:1}")"
60          ;;
61        +[[:alpha:]])
62          cmd="${cmd}$(add_plus "${1:1}")"
63          ;;
64        *)
65          echo "Invalid argument '$1'" >&2
66          exit 1
67          ;;
68      esac
69      shift
70    done

71    eval "$cmd | tee >(wc -l)"
```

Let's go over the interesting features of the script:

Lines 30–39

Defines two functions that are used to create single elements of the final pipeline. One function adds a filter that requires the specified letter, and the other function adds a filter that excludes any word that contains the specified letter.

Lines 40–51

Begins our processing of the positional parameters starting with the first one. We check if the first command line argument is the help option or the required five-character regular expression.

Line 52

We create the beginning of the pipeline. The first step is to filter the dictionary removing all words that are not exactly five characters long. Further we filter out out any word that contains a punctuation symbol (usually apostrophes) as well as uppercase letters since *Wordle* never uses proper nouns.

Line 56

Begins a loop that processes the rest of the command line arguments. Each time a *+letter* or *-letter* is found, we remove the leading plus or minus sign and call the respective function to add the next element of the pipeline to the command string.

Line 71

Here is where the actual work is performed. To get a count of the total number of words, we pipe the filtered word list into `tee`, which passes the list on to standard output (so that it gets displayed on the screen) and to a "file" that is actually the `wc` program running in a process substitution. Tricky.

Traps

In Chapter 10, we saw how programs can respond to signals. We can add this capability to our scripts too. While the scripts we have written so far have not needed this capability (because they have very short execution times and do not create temporary files), larger and more complicated scripts may benefit from having a signal handling routine.

When we design a large, complicated script, it is important to consider what happens if the user logs off or shuts down the computer while the script is running. When such an event occurs, a signal will be sent to all affected processes. In turn, the programs representing those processes can perform actions to ensure a proper and orderly termination of the program. Let's say, for example, that we wrote a script that created a temporary file during its execution. In the interests of good design, we would have the script delete the file when the script finishes its work. It would also be smart to have the script delete the file if a signal is received indicating that the program was going to be terminated prematurely.

`bash` provides a mechanism for this purpose known as a *trap*. Traps are implemented with the appropriately named builtin command, `trap`. `trap` uses the following syntax, where *`argument`* is a string that will be read and treated as a command and *`signal`* is the specification of a signal that will trigger the execution of the interpreted command:

```
trap argument signal [signal...]
```

Here is a simple example:

```
#!/bin/bash

# trap-demo: simple signal handling demo

trap "echo 'I am ignoring you.'" SIGINT SIGTERM

for i in {1..5}; do
    echo "Iteration $i of 5"
    sleep 5
done
```

This script defines a trap that will execute an `echo` command each time either the SIGINT or SIGTERM signal is received while the script is running. Execution of the program looks like this when the user attempts to stop the script by pressing CTRL-C:

```
[me@linuxbox ~]$ trap-demo
Iteration 1 of 5
Iteration 2 of 5
^CI am ignoring you.
Iteration 3 of 5
^CI am ignoring you.
Iteration 4 of 5
Iteration 5 of 5
```

As we can see, each time the user attempts to interrupt the program, the message is printed instead.

Constructing a string to form a useful sequence of commands can be awkward, so it is common practice to specify a shell function as the command. In this example, a separate shell function is specified for each signal to be handled:

```
#!/bin/bash

# trap-demo2: simple signal handling demo

exit_on_signal_SIGINT () {
    echo "Script interrupted." 2>&1
    exit 0
}

exit_on_signal_SIGTERM () {
    echo "Script terminated." 2>&1
    exit 0
}

trap exit_on_signal_SIGINT SIGINT
trap exit_on_signal_SIGTERM SIGTERM

for i in {1..5}; do
    echo "Iteration $i of 5"
```

```
    sleep 5
done
```

This script features two `trap` commands, one for each signal. Each trap, in turn, specifies a shell function to be executed when the particular signal is received. Note the inclusion of an `exit` command in each of the signal-handling functions. Without an `exit`, the script would continue after completing the function.

When the user presses CTRL-C during the execution of this script, the results look like this:

```
[me@linuxbox ~]$ trap-demo2
Iteration 1 of 5
Iteration 2 of 5
^CScript interrupted.
```

Besides the signals that are available to all programs, the `trap` builtin also supports several internal and bash-specific ones. Of particular interest are `EXIT` and `ERR`. As one might expect, the `EXIT` trap is activated when a script terminates. The `ERR` trap activates whenever a command (with certain exceptions) exits with a non-zero exit status. This works like a more useful version of the `set -e` option we looked at Chapter 30. Next, we have a short demonstration script of the `EXIT` and `ERR` traps. Notice the deliberate error on line number 5.

```
 1  #!/bin/bash

 2  # trap-demo3 - demonstrate ERR and EXIT signal handling

 3  trap "echo \"There is an error.\"" ERR
 4  trap "echo \"The program has ended.\"" EXIT

 5  echox "Running..."

 6  read -r -p "Say something... " something
 7  echo "$something"
```

This script defines the traps and then tries to display a line of text, but the `echo` command is misspelled. The script moves on to the next line where it waits for user input. Finally, it displays whatever the user entered.

When we run this script, we get the following:

```
[me@linuxbox ~]$ trap-demo3
./trap-demo3: line 8: echox: command not found
There is an error.
Say something...

The program has ended.
```

The first thing we get is an error message from the shell about our misspelled command. Next is the message from the `ERR` trap showing that it was activated. The `read` prompt appears next, and if we press ENTER, a blank line is output and the `EXIT` trap message announces that the program has ended.

TEMPORARY FILES

One reason signal handlers are included in scripts is to remove temporary files that the script may create to hold intermediate results during execution. There is something of an art to naming temporary files. Traditionally, programs on Unix-like systems create their temporary files in the */tmp* directory, a shared directory intended for such files. However, since the directory is shared, this poses certain security concerns, particularly for programs running with superuser privileges. Aside from the obvious step of setting proper permissions for files exposed to all users of the system, it is important to give temporary files nonpredictable filenames. This avoids an exploit known as a *temp race attack*. One way to create a nonpredictable (but still descriptive) name is to do something like this:

```
tempfile=/tmp/$(basename $0).$$.$RANDOM
```

This will create a filename consisting of the program's name, followed by its process ID (PID), followed by a random integer. Note, however, that the `$RANDOM` shell variable returns only a value in the range of 1–32,*767*, which is not a large range in computer terms, so a single instance of the variable is not sufficient to overcome a determined attacker.

A better way is to use the `mktemp` program (not to be confused with the `mktemp` standard library function) to both name and create the temporary file. The `mktemp` program accepts a template as an argument that is used to build the filename. The template should include a series of uppercase `X` characters, which are replaced by a corresponding number of random letters and numbers. The longer the series of `X` characters, the longer the series of random characters. Here is an example:

```
tempfile=$(mktemp /tmp/foobar.$$.XXXXXXXXXX)
```

This creates a temporary file and assigns its name to the variable `tempfile`. The `X` characters in the template are replaced with random letters and numbers so that the final filename (which, in this example, also includes the expanded value of the special parameter `$$` to obtain the PID) might be something like this:

```
/tmp/foobar.6593.UOZuvM6654
```

For scripts that are executed by regular users, it may be wise to avoid the use of the */tmp* directory and create a directory for temporary files within the user's home directory, with a line of code such as this:

```
[[ -d $HOME/tmp ]] || mkdir $HOME/tmp
```

Asynchronous Execution

It is sometimes desirable to perform more than one task at the same time. We have seen how all modern operating systems are at least multitasking if not multiuser as well. Scripts can be constructed to behave in a multitasking fashion.

Usually this involves launching a script that, in turn, launches one or more child scripts to perform an additional task while the parent script continues to run. However, when a series of scripts runs this way, there can be problems keeping the parent and child coordinated. That is, what if the parent or child is dependent on the other and one script must wait for the other to finish its task before finishing its own?

`bash` has a builtin command to help manage *asynchronous execution* such as this. The `wait` command causes a parent script to pause until a specified process (that is, the child script) finishes.

wait

We will demonstrate the `wait` command first. To do this, we will need two scripts. First, a parent script.

```
#!/bin/bash

# async-parent: Asynchronous execution demo (parent)

echo "Parent: starting..."

echo "Parent: launching child script..."
async-child &
pid=$!
echo "Parent: child (PID= $pid) launched."

echo "Parent: continuing..."
sleep 2

echo "Parent: pausing to wait for child to finish..."
wait "$pid"

echo "Parent: child is finished. Continuing..."
echo "Parent: parent is done. Exiting."
```

The second is a child script.

```
#!/bin/bash

# async-child: Asynchronous execution demo (child)

echo "Child: child is running..."
sleep 5
echo "Child: child is done. Exiting."
```

In this example, we see that the child script is simple. The real action is being performed by the parent. In the parent script, the child script is launched and put into the background. The process ID of the child script is recorded by assigning the `pid` variable with the value of the `$!` shell parameter, which will always contain the process ID of the last job put into the background.

The parent script continues and then executes a `wait` command with the PID of the child process. This causes the parent script to pause until the child script exits, at which point the parent script concludes.

When executed, the parent and child scripts produce the following output:

```
[me@linuxbox ~]$ async-parent
Parent: starting...
Parent: launching child script...
Parent: child (PID= 6741) launched.
Parent: continuing...
Child: child is running...
Parent: pausing to wait for child to finish...
Child: child is done. Exiting.
Parent: child is finished. Continuing...
Parent: parent is done. Exiting.
```

Named Pipes

In most Unix-like systems, it is possible to create a special type of file called a *named pipe*. Named pipes are used to create a connection between two processes and can be used just like other types of files. They are not that popular, but they're good to know about.

There is a common programming architecture called *client-server*, which can make use of a communication method such as named pipes, as well as other kinds of *interprocess communication* such as network connections.

The most widely used type of client-server system is, of course, a web browser communicating with a web server. The web browser acts as the client, making requests to the server, and the server responds to the browser with web pages.

Named pipes behave like files but actually form first-in, first-out (FIFO) buffers. As with ordinary (unnamed) pipes, data goes in one end and emerges out the other. With named pipes, it is possible to set up something like this

```
process1 > named_pipe
```

and this

```
process2 < named_pipe
```

and it will behave like this:

```
process1 | process2
```

Setting Up a Named Pipe

First, we must create a named pipe. This is done using the `mkfifo` command:

```
[me@linuxbox ~]$ mkfifo pipe1
[me@linuxbox ~]$ ls -l pipe1
prw-r--r-- 1 me   me    0 2009-07-17 06:41 pipe1
```

Here we use `mkfifo` to create a named pipe called `pipe1`. Using `ls`, we examine the file and see that the first letter in the attributes field is `p`, indicating that it is a named pipe.

Using Named Pipes

To demonstrate how the named pipe works, we will need two terminal windows (or alternately, two virtual consoles). In the first terminal, we enter a simple command and redirect its output to the named pipe.

```
[me@linuxbox ~]$ ls -l > pipe1
```

After we press ENTER, the command will appear to hang. This is because there is nothing receiving data from the other end of the pipe yet. When this occurs, it is said that the pipe is *blocked*. This condition will clear once we attach a process to the other end and it begins to read input from the pipe. Using the second terminal window, we enter this command:

```
[me@linuxbox ~]$ cat < pipe1
```

The directory listing produced from the first terminal window appears in the second terminal as the output from the `cat` command. The `ls` command in the first terminal successfully completes once it is no longer blocked.

Summing Up

Well, this completes our journey. The only thing left to do now is practice, practice, practice. Though we covered a lot of ground in our trek, there is so much more to explore. There are thousands of command line programs waiting to be discovered and enjoyed. Start digging around in */usr/bin* and see!

For readers still hungry for more, check out my follow-up book, *Adventures with the Linux Command Line*, available for free download at *https://linuxcommand.org*.